Stripe

This page owns platform Stripe provisioning, credentials, webhook registration, and account lifecycle. Config-plane authentication and revision writes are defined in config-plane transport; capability declaration syntax in features.

Supply and prerequisites

Platform-managed Stripe Checkout needs a server-capable application (`features.server: true`): Checkout creation uses a server secret, and the project serves the registered webhook. Enabling the integration supplies credentials and registers a callback; it does not implement checkout, fulfillment, or application persistence. An existing external payment link does not depend on this integration.

Cloud setup uses the current project' s promoted runtime binding and public Preview ingress; missing binding returns `runtime_binding_unavailable` . Local setup provisions test keys and skips webhook registration. The dev process need not be running to enable Stripe.

For Godot payment backends, follow backend publication before saving the first payment checkpoint. Stripe enable provisions the integration; it does not create that declaration.

Enable Stripe

Shopify and Stripe are **mutually exclusive per project**.

`POST <config-plane>/integrations/stripe/enable` with an empty body.
The session transport is `webdev.config` with method `POST` and path `integrations/stripe/enable` .

The operation is synchronous: eligibility check, Stripe sandbox provisioning, applicable webhook registration, and configuration storage complete within the request.

Success returns `{ok: true, revision}` and records `integrations: ["stripe"]` . It creates these platform-owned secret declarations at declaration `revision: 1` , with values supplied by the platform:

- `STRIPE_SECRET_KEY`
- `VITE_STRIPE_PUBLISHABLE_KEY`
- `STRIPE_WEBHOOK_SECRET`

The names and ownership are fixed; descriptions are editable. The `VITE_` prefix is the actual protocol name, not a requirement to use Vite. The publishable key is public; the secret and webhook keys are server secrets.

An already-enabled request is a no-op.
In a Manus session, the response can include `action_url`, a `manus-resource://` Stripe chat-status operation URI, not an HTTPS claim link. Local setup leaves `STRIPE_WEBHOOK_SECRET` unprovisioned until a Cloud Preview webhook is connected. This does not block development, but the first publish still requires it: connect a Cloud Preview webhook using the flow below before publishing. Never collect this platform-generated value through secure input or chat; if publish reports `required_secret_missing` for this key, connect the webhook instead.
Account status and claim information are available in Dashboard Settings › Integrations and the `stripe` field of `GET <config-plane>/config`.

Claiming the sandbox account later does not require a config change.

Test in the current Preview

Use `webdev.config`:

- `GET config`: read `stripe.sandbox_webhook_url` and `stripe.matches_current_preview`.
- `POST integrations/stripe/webhook`, body `{}`: connect the project's single test webhook to this session's Cloud Preview; replaces the previous endpoint and refreshes the runtime signing secret. Same-target retries are a no-op. Owner only; no URL argument.

For local work, tell the user full webhook testing needs a temporary Cloud Sandbox. If requested, sync the project and switch with user approval, start its Preview, then connect the webhook. After a sandbox/session change, read config before testing and reconnect if needed. Dashboard automatic re-registration remains active for Cloud previews; Local skips it. Another Cloud session may replace the target. Registration and URL matching do not prove successful payment delivery; test the application handler.

Failure and revision semantics

Successful setup advances the config revision. A failed setup leaves the config document unchanged and returns a reason code. An open secure-input operation blocks enable with `409 revision_conflict`, `reason_code: secret_input_in_progress`; this is an input-operation conflict, not proof that the config revision changed. Config-plane operation state owns that conflict's lifecycle.

Unsupported regions return a setup failure rather than a provisioned account. Settings › Integrations is the owner's recovery surface. The Secrets page displays the three managed values read-only and links to that surface.

Registered webhook contract

The platform registers `POST /api/stripe/webhook`. Registration does not implement that application handler. Its subscription is fixed to these six events:

- `payment_intent.succeeded`
- `payment_intent.payment_failed`
- `customer.subscription.created`
- `customer.subscription.updated`
- `customer.subscription.deleted`
- `checkout.session.completed`

This integration does not expose an operation to add or remove subscribed event types. Other event handlers therefore do not gain delivery through this subscription.

Stripe signature verification uses `STRIPE_WEBHOOK_SECRET` and the exact request bytes; parsing and re-serializing JSON changes those bytes. Webhooks may be redelivered. A Checkout return URL is navigation, not evidence of payment, and webhook receipt is separate from application fulfillment. The platform supplies no application entitlement store or deduplication implementation.

Account lifecycle and go-live

Sandbox claim information is in Settings › Integrations. After the site is published, the owner supplies live publishable and secret keys as a pair in that same Dashboard section. The platform then points the production webhook at the published domain. Republish is required for the deployed application to receive the live values. Live keys are not config-document fields.

Payment implementation and completion

Read this payment contract before modifying Stripe configuration or implementing platform-managed checkout. For a new project needing that capability, declare the server at creation. Read the configuration/secret contracts before revising managed declarations; use the single enable operation, never hand-add its fixed keys or create a second webhook in the Stripe Dashboard. Do not ask the user to provide the sandbox keys that the platform provisions.

Design payment persistence from the start. Use the server project's managed database unless the user selected another durable store or explicitly chose a staged design without one. Stripe owns the money; the application must store what the buyer is entitled to and how it is fulfilled. If the user explicitly chooses no durable storage, explain before proceeding which requested guarantees cannot be supplied: repeat access, purchase

records and reliable event deduplication. Do not pretend that “the platform does not implement it” completes this application work.

Durable application records

Persist the fulfillment/entitlement record, the associated application user/customer identity and required business metadata. Keep the Stripe identifiers needed later: customer ID, active subscription ID, payment-intent ID, invoice ID when tracking subscription invoices, and product/price IDs when offline mapping is required. Record processed event IDs and timestamps so redelivery can be recognized.

Do not mirror Stripe's data without a concrete requirement. Fetch amounts, currency/status, payment-method display details, subscription periods/cancellation, receipt URLs, billing details, prices and tax information from Stripe; cache a field only for a specific reporting or performance need and record why. Start with a minimal schema and add fields for stated requirements. Make schema setup deterministic, repeatable and additive, because the development and published application share data.

Never persist card numbers, security codes or expiry dates. Do not persist raw webhook payloads, API keys or client secrets as application records. Use the protected credential flow and the smallest business record needed for delivery and deduplication.

Checkout and buyer identity

Create Checkout Sessions on the server and return their URL to the frontend. Prefill the authenticated buyer's email with `customer_email`; include `client_reference_id` and `metadata.user_id` linked to the application user, plus `metadata.customer_email` and `metadata.customer_name` when available. This ties `checkout.session.completed` to the correct buyer. Do not treat their status as optional platform fields as permission to omit the application's required buyer linkage.

Set `allow_promotion_codes: true`. If using a Stripe SDK, pass its parameters as values without pre-encoding them again. Build `success_url` and `cancel_url` from the actual browser-visible application origin, not an internal proxy Host or a guessed environment URL. Grant access only from stored fulfillment state, never from user-editable redirect parameters.

Centralize products and prices in one application module; `products.ts` is the TypeScript example, not a required filename for other stacks. Open checkout in a new browser tab and show a toast explaining the transition.

Webhook processing

Implement the registered `/api/stripe/webhook` route for the six subscribed event types. Acknowledge subscribed events the application does not handle rather than returning an error that causes repeated delivery. Verify each event's signature before trusting or storing

it. The handler must receive the exact unparsed bytes; do not place a JSON parser ahead of signature verification.

Once an accepted event has been recorded, acknowledge promptly and perform slower work afterward. Check the durable processed-event record so the second delivery is a no-op: fulfillment must be idempotent. Save the essential identifiers and application state, not the raw payload. When a Stripe API version changes an entity/field, map the logic to its current equivalent identifier.

Setup and verification sequence

After enable changes the config revision, read the current revision before the next configuration write. A setup failure needs its specific `reason_code` : settle or withdraw pending secret input instead of repeatedly rereading/resubmitting a revision. If the supported-region setup cannot provision an account, report that reason and continue the provider setup only through the owner's supported Dashboard/key path.
After provisioning, start the project's dev server. The server need not already be running for the enable operation itself.

Provide an `/orders` or `/payments` history view showing completed purchases with date, amount, status and items. Represent cancellation and still-processing returns separately; returning before the webhook arrives is not by itself a payment failure.

Instruct the user to test payments with Stripe's `4242 4242 4242 4242` test card. Stripe requires a minimum of $0.50 for USD payments; respect the selected currency/provider's amount constraints.

Remind the owner to claim the Stripe sandbox promptly; the claim can still happen after development has begun.

For failed callbacks, inspect the Stripe Dashboard delivery result. When every signature verification fails, check preservation of the raw body before assuming the platform supplied a wrong secret. Keep the already-documented owner-controlled live-key and republish flow.

Shopify

Shopify is a separate managed integration. Read Shopify before choosing a stack or initializing a new Shopify storefront, and before enabling Shopify or writing storefront/Admin API code in an existing project. Its page owns storefront architecture, store selection, and the Admin-tool boundary.